

浅层神经网络

学习目标

浅层神经网络表示

单个样本的向量化表示

激活函数的选择

为什么需要非线性的激活函数

修改激活函数的前向传播和反向传播

代码实现

定义神经网络结构

初始化模型参数

循环中的第一步：向前传播

循环中的第二步：计算损失

循环中的第三步：反向传播

循环中第四步：更新梯度

建立网络模型训练逻辑

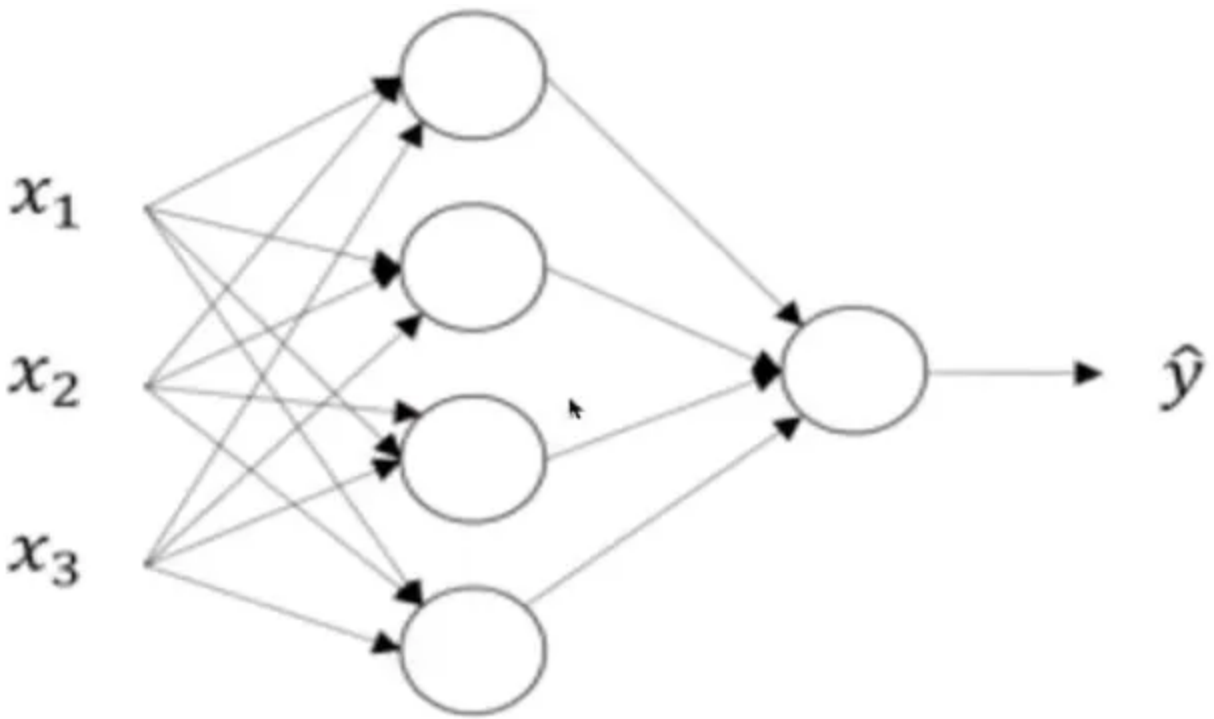
预测结果

学习目标

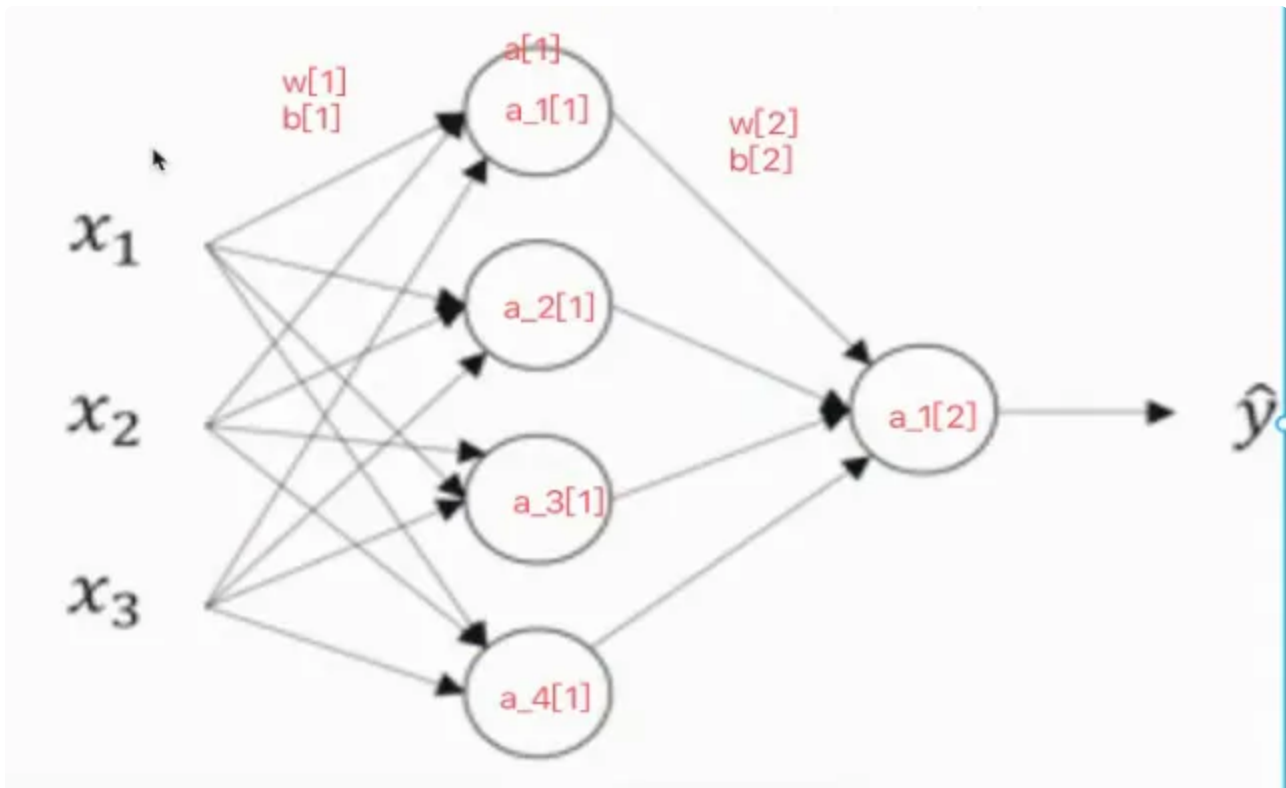
- 目标
 - 知道浅层神经网络的前向计算过程
 - 知道选择激活函数的原因
 - 说明浅层神经网络的反向传播推导过程
- 应用
 - 应用完成一个浅层神经网络结构进行分类

浅层神经网络表示

之前已经说过神经网络的结构，假设我们有如下结构的网络



对于这个网络我们建立一个简单的图示？我们对第一个隐藏层记为[1],输出层记为[2], 如下图



计算图如下



- 第一层中的第一个神经元

$$z_1^{[1]} = (W_1^{[1]})^T x + b_1^{[1]}$$

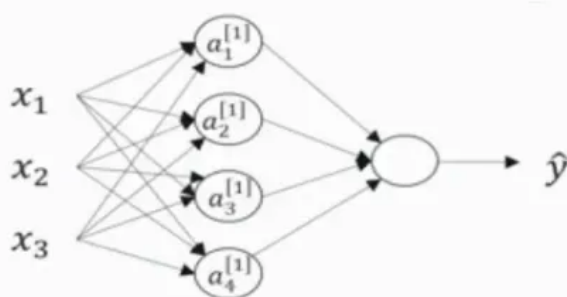
$$a_1^{[1]} = \sigma(z_1^{[1]})$$

- 第一层中的第一个神经元

$$z_2^{[1]} = (W_2^{[1]})^T x + b_2^{[1]}$$

$$a_2^{[1]} = \sigma(z_2^{[1]})$$

得出第一层的计算：



$$\begin{aligned} z_1^{[1]} &= w_1^{[1]T} x + b_1^{[1]}, a_1^{[1]} = \sigma(z_1^{[1]}) \\ z_2^{[1]} &= w_2^{[1]T} x + b_2^{[1]}, a_2^{[1]} = \sigma(z_2^{[1]}) \\ z_3^{[1]} &= w_3^{[1]T} x + b_3^{[1]}, a_3^{[1]} = \sigma(z_3^{[1]}) \\ z_4^{[1]} &= w_4^{[1]T} x + b_4^{[1]}, a_4^{[1]} = \sigma(z_4^{[1]}) \end{aligned}$$

单个样本的向量化表示

隐藏层：四个神经元，输出层：一个神经元

把上面第一层的计算过程用更加简单的形式表现出来

$$\begin{pmatrix} \dots & (W_1^{[1]})^T & \dots \\ \dots & (W_2^{[1]})^T & \dots \\ \dots & (W_3^{[1]})^T & \dots \\ \dots & (W_4^{[1]})^T & \dots \end{pmatrix} * \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{pmatrix}$$

那么对于刚才我们所举的例子，将所有层通过向量把整个前向过程表示出来，并且确认每一个组成部分的形状

前向过程计算：

$$z^{[1]} = W^{[1]}x + b^{[1]} \text{ 形状: } (4,1) = (4,3) * (3,1) + (4,1)$$

$$a^{[1]} = \sigma(z^{[1]}) \text{ 形状: } (4,1)$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \text{ 形状: } (1,1) = (1,4) * (4,1) + (1,1)$$

$$a^{[2]} = \sigma(z^{[2]}) \text{ 形状: } (1,1)$$

那么如果有多个样本，需要这样做

for i = 1 to m

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

多个样本向量化表示

假设一样含有M个样本，那么上述过程变成

$$Z^{[1]} = W^{[1]}X + b^{[1]} \text{ 形状: } (4,m) = (4,3) * (3,m) + (4,1)$$

$$A^{[1]} = \sigma(Z^{[1]}) \text{ 形状: } (4,m)$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} \text{ 形状: } (1,m) = (1,4) * (4,m) + (1,1)$$

$$A^{[2]} = \sigma(Z^{[2]}) \text{ 形状: } (1,m)$$

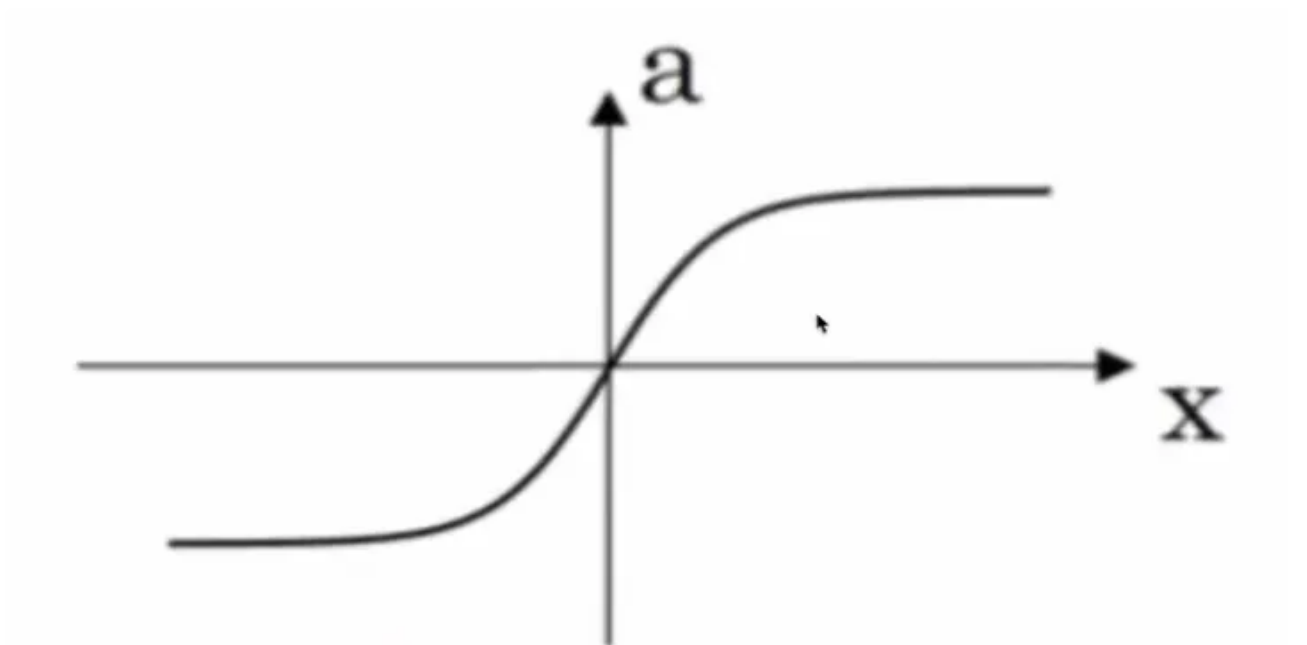
注意：每一层计算的时候形状的变化

激活函数的选择

涉及到网络的优化时候，会有不同的激活函数选择有一个问题是神经网络的隐藏层和输出单元用什么激活函数。之前我们都是选用sigmoid函数，但有些时候其他函数的效果会好得多，大多数通过实践得来，没有很好的解释性。

可供选择的激活函数有：

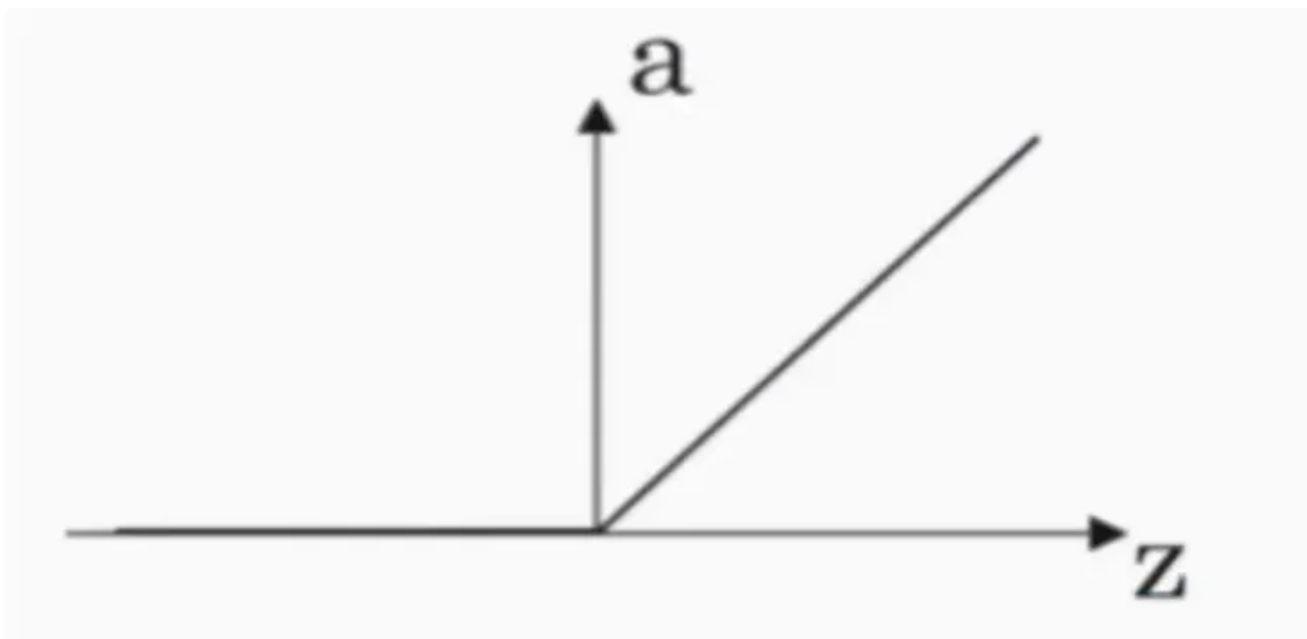
- tanh函数（双曲正切函数）：



效果比sigmoid函数好，因为函数输出介于-1和1之间。

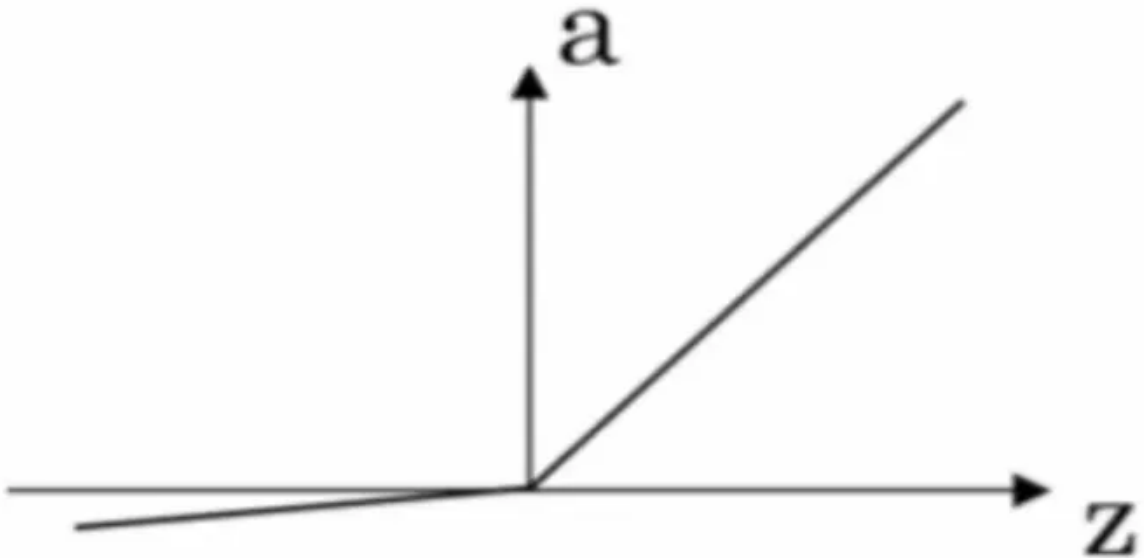
注：tanh函数存在和sigmoid函数一样的缺点：当 z 趋紧无穷大（或小），导数的梯度（斜率）就趋于0，这使得梯度算法的速度会变慢。

- ReLU函数（修正线性函数）



当 z 大于0的时候，梯度始终为1，从而提高神经网络基于梯度算法的运算速度，收敛速度远大于sigmoid和tanh。然而当 z 小于等于0时，梯度一直为0，但是实际的运用中，该缺陷的影响不是很大。

- Leaky ReLU（带泄漏的ReLU）



Leaky ReLU保证在 $z < 0$ 的时候，梯度仍然不为0.理论上说，Leaky ReLU有ReLU的所有优点，但在实际操作中没有证明总是好于ReLU,因此不常用

为什么需要非线性的激活函数

使用线性激活函数和不使用激活函数、直接使用Logistic回归没有区别，那么无论神经网络有多少层，输出都是输入的线性组合，与没有隐藏层效果相当，就成了最原始的感知器

$$a[1] = z[1] = W[1]x + b[1]$$

$$a[2] = z[2] = W[2]a[1] + b[2]$$

那么这样的话相当于

$$a[2] = z[2] = W[2](W[1]x + b[1]) + b[2] = (W[2]W[1])x + (W[2]b[1] + b[2]) = wx + b$$

修改激活函数的前向传播和反向传播

将上述网络的隐层激活函数修改为tanh，最后一层同样还是二分类，所以激活函数选择依然是sigmoid函数

- 前向传播

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \tanh(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

- 反向梯度下降

那么通过这个计算图来理解这个过程，单个样本的导数推导过程：



由于网络已经存在两层，所以我们需要从后往前得到导数结果，并且多个样本的情况改写成

最后一个输出层的参数的导数：

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1)$$

隐藏层的导数计算：

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * (1 - g(Z^{[1]})^2) = W^{[2]T} dZ^{[2]} * (1 - A^{[1]})^2$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1)$$

分为两个部分进行反向传播

- 最后一层的梯度计算： $dw[2], d[2]$
- 隐层的梯度计算： $dw[1], db[1]$

代码实现

定义神经网络结构

```
def layer_sizes(X, Y):
    """
    """
    # 输入层大小
    n_x = X.shape[0]
    # 隐层大小
    n_h = 4
    # 输出层大小
    n_y = Y.shape[0]
    ### END CODE HERE ###
    return (n_x, n_h, n_y)
```

初始化模型参数

随机初始化权重以及偏置为0

```
def initialize_parameters(n_x, n_h, n_y):
    """
    输入每层的神经元数量

    返回：隐层、输出层的参数
    """

    np.random.seed(2)

    ### 开始
    # 创建隐层的两个参数
    # 让值小一些
    W1 = np.random.randn(n_h, n_x) * 0.01
    b1 = np.zeros((n_h, 1))

    # 创建输出层前对应的参数
    W2 = np.random.randn(n_y, n_h) * 0.01
    b2 = np.zeros((n_y, 1))

    ### 结束

    # 检测维度是否符合要求
    assert (W1.shape == (n_h, n_x))
    assert (b1.shape == (n_h, 1))
    assert (W2.shape == (n_y, n_h))
    assert (b2.shape == (n_y, 1))

    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2}

    return parameters
```

这样的我们就初

循环中的第一步：向前传播

根据给的公式，完成该函数

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \tanh(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

使用函数：np.dot , np.tanh , np.sigmoid,其中sigmoid函数是自定义的

```
1]: def sigmoid(x):  
    """  
    """  
    s = 1/(1+np.exp(-x))  
    return s
```

```

def forward_propagation(X, parameters):
    """
    Argument:
    X:(n_feature, m)

    Returns:
    A2:最后一层的输出
    cache:用于反向传播计算的存储中间计算结果的字典
    """
    # 获取参数
    ### 开始
    # 取出每一层的参数
    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['b2']

    # 进行一层一层的运算
    Z1 = np.dot(W1, X) + b1
    A1 = np.tanh(Z1)

    # 第二层
    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2)
    ### 结束

    assert(A2.shape == (1, X.shape[1]))

    cache = {"Z1": Z1,
             "A1": A1,
             "Z2": Z2,
             "A2": A2}

    return A2, cache

```

取

循环中的第二步：计算损失

完成损失计算的过程，根据损失公式

$$J = -\frac{1}{m} \sum_{i=0}^m \left(y^{(i)} \log(a^{[2](i)}) + (1 - y^{(i)}) \log(1 - a^{[2](i)}) \right)$$

设计多个维度，使用np.multiply进行乘法运算

```
def compute_cost(A2, Y, parameters):
    """
    parameters: 最后一层输出, 目标值, 参数
    return: 损失
    """
    m = Y.shape[1]
    ### 开始
    logpro = np.multiply(np.log(A2), Y) + np.multiply((1 - Y), (np.log(1 - A2)))
    cost = - 1 / m * np.sum(logpro)

    ### 结束

    cost = np.squeeze(cost)

    assert(isinstance(cost, float))

    return cost
```

循环中的第三步：反向传播

反向传播在这个网络中分为两步 第一次计算：

$$dZ^{[2]} \stackrel{!}{=} A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1)$$

隐层的反向传播梯度计算：

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * (1 - g(Z^{[1]})^2) = W^{[2]T} dZ^{[2]} * (1 - A^{[1]})^2$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1)$$

```

In [26]: def backward_propagation(parameters, cache, X, Y):
    """
    parameters:
    cache: 存储每层前向传播计算结果
    x: 数据特征
    y: 数据目标值

    return: 每个参数的梯度
    """
    # 得出训练样本数量
    m = X.shape[1]

    # 获取参数和缓存中的输出
    ### 开始
    w1 = parameters['w1']
    w2 = parameters['w2']
    A1 = cache['A1']
    A2 = cache['A2']
    ### 结束

    # 反向传播计算梯度
    ### 开始
    # 最后一层的参数梯度计算
    dz2 = A2 - Y
    dw2 = 1/m * np.dot(dz2, A1.T)
    db2 = 1/m * np.sum(dz2, axis=1)

    # 隐藏层的参数梯度计算
    dz1 = np.dot(w2.T, dz2) * (1 - np.power(A1, 2))
    dw1 = 1/m * np.dot(dz1, X.T)
    db1 = 1/m * np.sum(dz1, axis=1)
    ### 结束

    grads = {"dw1": dw1,
              "db1": db1,
              "dw2": dw2,
              "db2": db2}

    return grads

```

循环中第四步：更新梯度


```

def update_parameters(parameters, grads, learning_rate = 0.005):
    """
    参数：网络参数，梯度，学习率
    返回更新之后的参数
    """
    # 获取参数以及梯度
    ### 开始
    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['W2']

    dW1 = grads['dW1']
    db1 = grads['db1']
    dW2 = grads['dW2']
    db2 = grads['db2']

    ## 结束

    # 使用学习率更新参数
    ### 开始
    W1 = W1 - learning_rate * dW1
    b1 = b1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    b2 = b2 - learning_rate * db2

    ### 结束

    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2}

    return parameters

```

建立网络模型训练逻辑

```

def nn_model(X, Y, num_iterations = 10000, print_cost=False):
    """
    """

    np.random.seed(3)
    n_x = layer_sizes(X, Y)[0]
    n_y = layer_sizes(X, Y)[2]

    # 初始化参数
    ### 开始
    # 获取网络的层大小
    # 2, 4, 1
    n_x, n_h, n_y = layer_sizes(X, Y)
    parameters = initialize_parameters(n_x, n_h, n_y)
    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['W2']

    ### 结束

```

```

# 循环
for i in range(0, num_iterations):

    ### 开始
    # 前向传播
    A2, cache = forward_propagation(X, parameters)

    # 计算损失
    cost = compute_cost(A2, Y, parameters)

    # 反向传播计算梯度
    grads = backward_propagation(parameters, cache, X, Y)

    # 利用梯度更新参数
    parameters = update_parameters(parameters, grads)

    ### 结束

    if i % 1000 == 0:
        print ("迭代次数 %i: %f" % (i, cost))

return parameters

```

预测结果

```

def predict(parameters, X):
    """
    """

    # 计算概率值, 以及判断类别
    A2, cache = forward_propagation(X, parameters)
    predictions = np.array( [1 if x > 0.5 else 0 for x in A2.reshape(-1,1)] ).reshape(A2.shape)

    return predictions

```

```

# 测试
parameters = nn_model(X, Y, num_iterations = 10000)

predictions = predict(parameters, X)
print ('准确率: %d' % float((np.dot(Y,predictions.T) + np.dot(1-Y,1-predictions.T))/float(Y.size)*100) + '%')

```